

Java – Créer une API REST avec Spring Boot

Spring Boot est un framework Java qui permet de créer rapidement des applications web et des API REST robustes.

Introduction à Spring Boot

Spring Boot simplifie le développement avec Spring en fournissant :

- Une configuration automatique
- Un serveur embarqué (Tomcat)
- Une structure de projet standard

Objectif principal :

- Démarrer rapidement un projet sans configuration complexe

Spring Boot est largement utilisé pour :

- Les API REST
- Les microservices
- Les applications backend

Pourquoi les API REST sont importantes

Une API REST permet à des applications de communiquer entre elles via HTTP.

Avantages :

- Séparation front-end / back-end
- Communication avec applications web, mobiles ou jeux
- Format standard basé sur JSON
- Scalabilité et maintenabilité

Exemple d'utilisation :

- Un client demande des données
- Le serveur répond avec du JSON

Création du projet Spring Boot

La création du projet se fait via **Spring Initializr**.

Paramètres essentiels :

- Project : Maven
- Language : Java
- Spring Boot : version stable
- Packaging : Jar
- Dependencies :
 - Spring Web

Note importante :

- La version de Java doit être compatible avec Spring Boot
(exemple courant : Java 17)

Structure minimale d'un projet Spring Boot

Éléments clés :

- Classe principale avec `@SpringBootApplication`
- Dossier `controller` pour les API
- Dossier `model` pour les objets métier

Spring Boot démarre l'application via une méthode `main`.

Création d'un REST Controller

Un contrôleur REST expose des endpoints accessibles via HTTP.

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Spring Boot";
    }
}
```

- `@RestController` : indique un contrôleur REST
- `@GetMapping` : route HTTP GET

Retourner du JSON (objet Java)

Spring Boot convertit automatiquement un objet Java en JSON.

Objet Java :

```
public class Joueur {  
    public String nom;  
    public int score;  
  
    public Joueur(String nom, int score) {  
        this.nom = nom;  
        this.score = score;  
    }  
}
```

Endpoint REST retournant un objet

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
public class JoueurController {  
  
    @GetMapping("/joueur")  
    public Joueur getJoueur() {  
        return new Joueur("Alice", 150);  
    }  
}
```

Résultat côté client (JSON) :

```
{  
  "nom": "Alice",  
  "score": 150  
}
```

Paramètres d'URL (Path et Query)

Spring Boot permet de récupérer des paramètres directement depuis l'URL.

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class JoueurController {

    @GetMapping("/joueur/{nom}")
    public Joueur getJoueur(@PathVariable String nom) {
        return new Joueur(nom, 100);
    }
}
```

Exemple d'appel :

```
GET /joueur/Alice
```

Paramètre de requête (Query Parameter)

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;  
  
@GetMapping("/score")  
public Joueur getScore(@RequestParam int score) {  
    return new Joueur("Joueur", score);  
}
```

Exemple d'appel :

```
GET /score?score=200
```


Request Body (données envoyées par le client)

Le `Request Body` permet d'envoyer un objet JSON au serveur.

Endpoint avec POST

```
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@PostMapping("/joueur")
public Joueur creerJoueur(@RequestBody Joueur joueur) {
    return joueur;
}
```

Spring Boot :

- Déséréalise automatiquement le JSON en objet Java
- Sériálise l'objet retourné en JSON

Exemple de JSON envoyé par le client

```
{  
  "nom": "Bob",  
  "score": 300  
}
```

Conclusion

- Spring Boot accélère le développement backend
- Les API REST sont essentielles pour les architectures modernes
- Les objets Java sont automatiquement sérialisés en JSON
- Un contrôleur REST permet d'exposer facilement des données